Notas de Aula
Prof. Adriano Milanez

Programação Web I
Responsive Web Development
Front-End Design Engineering
Programação Web Responsiva

[aula 22]
[Atualizado em: 30/09/2025]

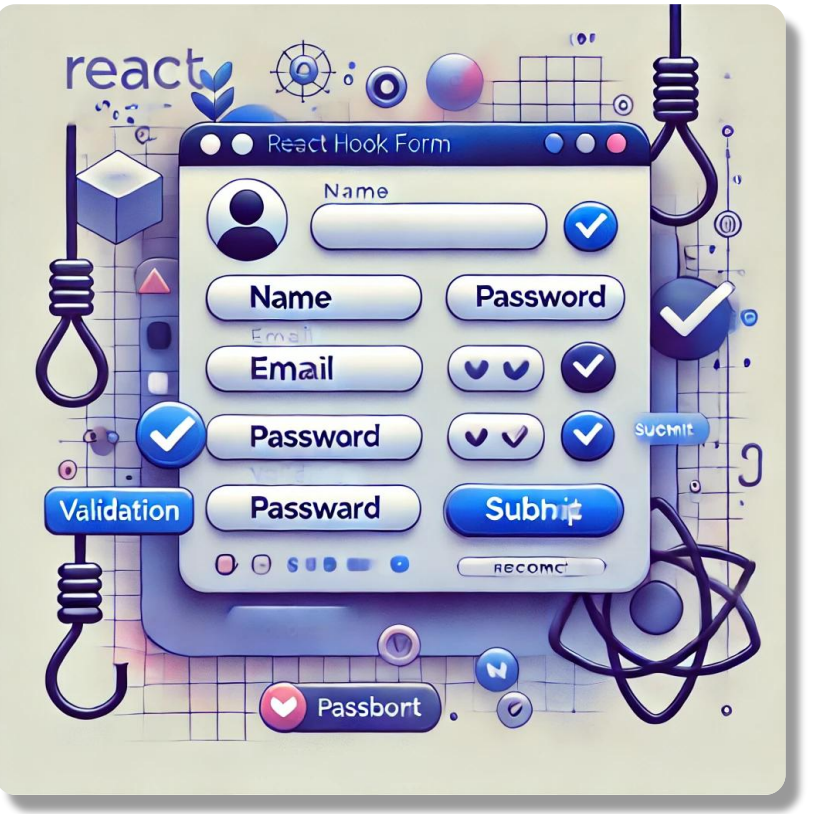
[VITE – React Hook Form]

1

VITE – REACT HOOK FORM

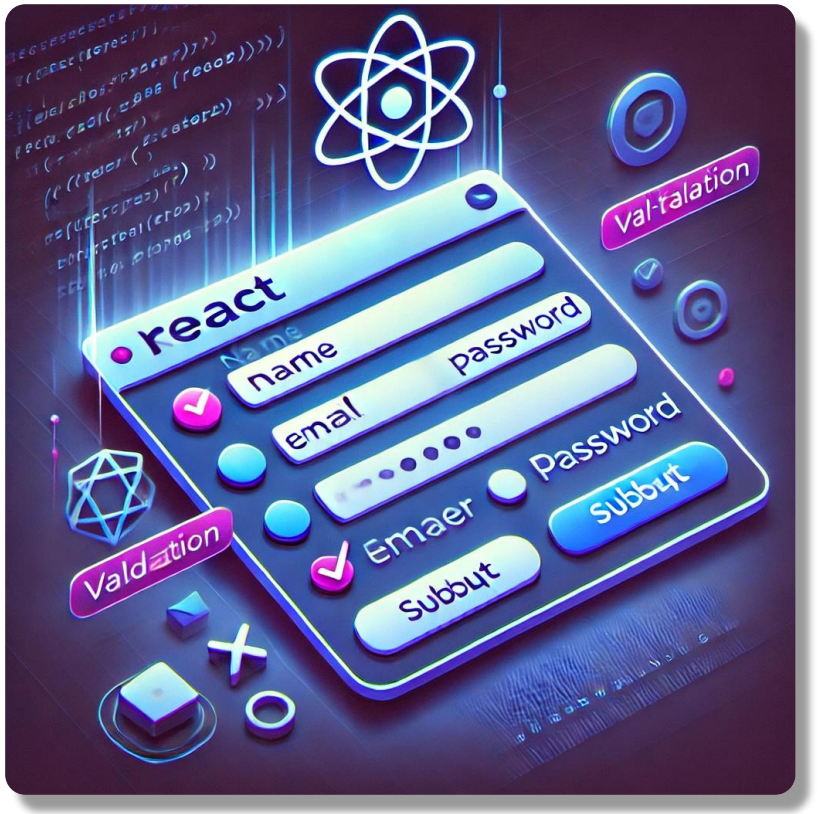
VALIDAÇÃO RÁPIDA E PRECISA

- ✓ O **React Hook Form** é uma biblioteca para gerenciamento de formulários no React que se destaca pela **performance** e **simplicidade**.
- ✓ Ele reduz re-renderizações desnecessárias, melhora a validação e facilita a integração com outras bibliotecas, como **Yup** e **Zod**.
- ✓ O **React Hook Form** é uma biblioteca React que oferece uma abordagem simples e eficiente para lidar com formulários. Ele utiliza **hooks** do React para gerenciar o estado do formulário, lidar com validação e enviar dados.
- ✓ É uma biblioteca que auxiliará a organizar e padronizar as validações dos formulários por toda a aplicação.



VALIDAÇÃO RÁPIDA E PRECISA

- ✓ **Por que usar o React Hook Form?**
  - ✓ **Menos re-renderizações**: Ele trabalha diretamente com *refs* do React, evitando que o estado do formulário cause re-renderizações desnecessárias.
  - ✓ **Validação simplificada**: Suporte nativo ao *register* e fácil integração com bibliotecas como Yup/Zod.
  - ✓ **Menos código boiler plate**: Comparado ao *useState* e *useEffect*, o código fica mais limpo.
  - ✓ **Melhor performance**: Ideal para formulários grandes, pois não depende do estado global do React.



VALIDAÇÃO RÁPIDA E PRECISA

- ✓ **Principais HOOKS fornecidos pelo Form Hooks:**
  - ✓ Hook Principal
    - ✓ useForm
      - ✓ é o **hook principal do React Hook Form**. Ele é responsável por **gerenciar todo o estado do formulário** de maneira simples e eficiente, sem precisar criar estados separados para cada campo.
      - ✓ Ele retorna funções e métodos importantes, como:
        - ✓ **register**: é uma função que registra os campos do formulário para o gerenciamento interno.
        - ✓ **handleSubmit**: função que processa o envio do formulário.
        - ✓ **reset**: é uma função que redefine os campos do formulário para seus valores iniciais.
        - ✓ **formState**: é uma propriedade com informações sobre erros (errors), estado de submissão (isSubmitting), etc.reset → permite limpar ou redefinir os valores do formulário.
        - ✓ **errors**: é uma propriedade do objeto formState que contém informações sobre os erros de validação de cada campo.

VALIDAÇÃO RÁPIDA E PRECISA

- **Principais HOOKS fornecidos pelo Form Hooks:**
  - **Validações suportadas são**:
    - **min:** Esta opção define o valor mínimo aceitável para um campo numérico. Se o valor do campo for menor que o valor mínimo especificado, a validação falhará e a mensagem de erro associada será exibida.
    - **max:** Similar ao min, esta opção define o valor máximo aceitável para um campo numérico. Se o valor do campo for maior que o valor máximo especificado, a validação falhará e a mensagem de erro associada será exibida.
    - **maxlength:** Esta opção define o número máximo de caracteres permitidos para um campo de texto. Se o número de caracteres exceder o limite especificado, a validação falhará e a mensagem de erro associada será exibida.
    - **minlength:** Similar ao maxlength, esta opção define o número mínimo de caracteres permitidos para um campo de texto. Se o número de caracteres for menor que o limite especificado, a validação falhará e a mensagem de erro associada será exibida.

VALIDAÇÃO RÁPIDA E PRECISA

- **Introdução ao Hook Form**
  - **Exemplo 01 – Formulário simples**

```
npx create-vite-app@latest
```

  - Na sequência, vamos instalar a biblioteca HOOK FORM

```
npm install react-hook-form
```

VALIDAÇÃO RÁPIDA E PRECISA

- **Introdução ao Hook Form**

  - **Exemplo 01 – Formulário simples**
    - Vamos criar um formulário bem simples, com dois campos para fazermos suas respectivas validações: nome e sobrenome

VALIDAÇÃO RÁPIDA E PRECISA

1

- ✓ **Introdução ao Hook Form**
  - ✓ **Exemplo 01 – Formulário simples**
    - ✓ Vamos criar um formulário bem simples, com dois campos para fazermos suas respectivas validações: nome e sobrenome.

```
1.  //src/Exemplo01/Exemplo01.tsx
2.  const Exemplo01 = () => {
3.  return (
4.   <>
5.    <h1>Exemplo 01 - Formulário com React Hook Form</h1>
6.    <h2>Exemplo 01 - Formulário com React Hook Form</h2>
7.    <form>
8.      <div>
9.       <label htmlFor="nome">Nome: </label> <input />
11.     </div>
12.     <div>
13.      <label htmlFor="sobrenome">Sobrenome: </label>
         <input />
15.     </div>
16.     <button type="submit">Enviar</button
17.   </form>
18.  </>
19.  )
20. }
21. export default App
```

VALIDAÇÃO RÁPIDA E PRECISA

- **Introdução ao Hook Form**

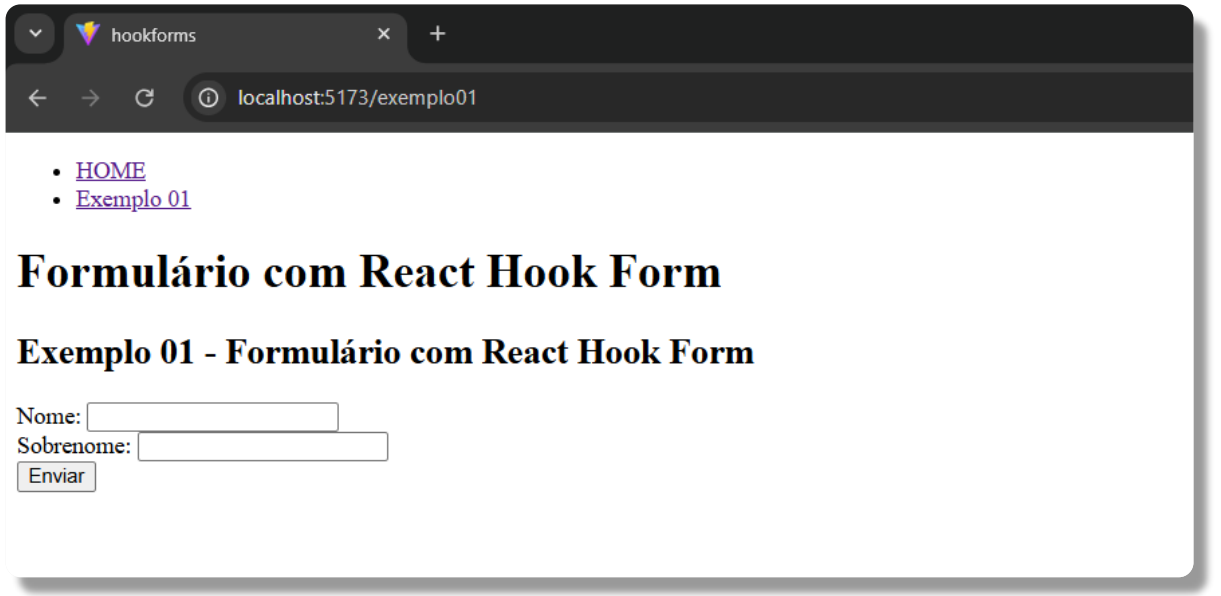
  - **Exemplo 01 – Formulário simples**

    - Vamos instalar o REACT-ROUTER-DOM e modificar o APP para conseguirmos acessar nosso Exemplo01.

VALIDAÇÃO RÁPIDA E PRECISA

- ✓ **Introdução ao Hook Form**

  - ✓ **Exemplo 01 – Formulário simples**
    - ✓ Vamos criar um formulário bem simples, com dois campos para fazermos suas respectivas validações: nome e sobrenome.



VALIDAÇÃO RÁPIDA E PRECISA

✓ **Introdução ao Hook Form** - Exemplo 01 – Vamos modificar o arquivo que acabamos de criar

```
1.  //src/Exemplo01/Exemplo01.tsx
2.  import { useForm, SubmitHandler } from 'react-hook-form';
    // Esta linha importa os hooks useForm e SubmitHandler da biblioteca react-hook-form. Esses hooks são usados para
    gerenciar o estado do formulário e lidar com a submissão do formulário, respectivamente.
3.
4.  type FormValues = {
5.   firstName: string;
6.   lastName: string;
7.  }; // Esta linha define um tipo TypeScript FormValues, que representa a estrutura dos dados do formulário. Neste
    caso, o formulário possui dois campos: firstName e lastName, ambos do tipo string.
8.
9.  const Exemplo01 = () => {
10.
11.  const { register, handleSubmit, formState: { errors } } = useForm<FormValues>();
     // Utiliza o hook useForm para inicializar o estado do formulário. Retorna várias funções e objetos, como register
     para registrar campos do formulário, handleSubmit para lidar com a submissão do formulário e errors para armazenar
     erros de validação do formulário.
12.  const onSubmit: SubmitHandler<FormValues> = (data) => {
13.   console.log(data);
14.  } // Define uma função chamada onSubmit, que será chamada quando o formulário for submetido. Ela recebe os dados do
     formulário como argumento e, neste caso, simplesmente os imprime no console.
```

VALIDAÇÃO RÁPIDA E PRECISA

- ✓ **Introdução ao Hook Form**
  - ✓ Exemplo 01 – Analisando linha a linha o arquivo page.tsx

```
15.
16.   return (
17.    <>
18.      <h1>Exemplo 01 - Formulário com React Hook Form</h1>
19.      <h2>Exemplo 01 - Formulário com React Hook Form</h2>
20.      <form onSubmit={handleSubmit(onSubmit)}>
         // Esta parte do código retorna o JSX (JavaScript XML) que representa o componente Home. Ele contém um <h1> com o
         texto "Formulário com React Hook Form" e um formulário. O evento onSubmit do formulário está vinculado à função
         handleSubmit, que irá executar a função onSubmit quando o formulário for submetido.
21.        <div>
22.            <label>Primeiro Nome:</label>
23.            <input {...register("firstName", { required: true })} />
               // Essa parte do código representa um campo de entrada para o "Primeiro Nome". O atributo register é usado para
               registrar este campo no formulário. O objeto passado para register contém configurações de validação, neste
               caso, indicando que o campo é obrigatório. Se houver um erro de validação para este campo (errors.firstName),
               será renderizado um <span> com a mensagem "Este campo é obrigatório"
24.            {errors.firstName && <span>Este campo é obrigatório</span>}
25.        </div>
26.
```

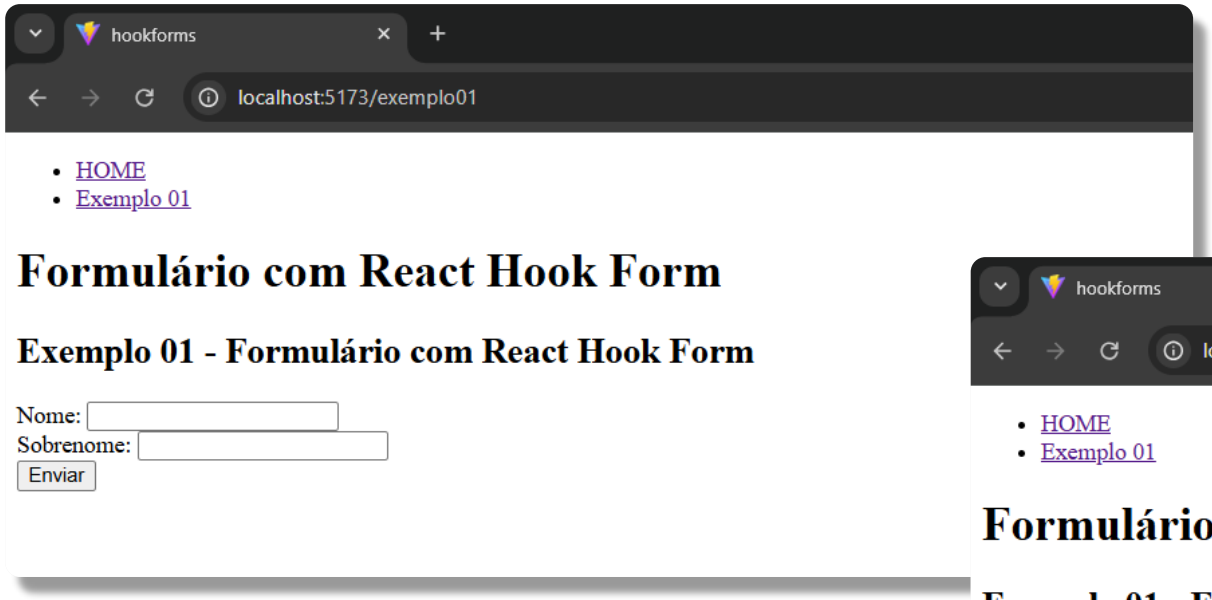
VALIDAÇÃO RÁPIDA E PRECISA

- ✓ **Introdução ao Hook Form**
  - ✓ Exemplo 01 – Analisando linha a linha o arquivo page.tsx

```
27.         <div>
28.             <label>Sobrenome:</label>
29.             <input {...register("lastName", { required: true })} />
                // Essa parte do código representa um campo de entrada para o "Sobrenome", similar ao campo anterior.
30.             {errors.lastName && <span>Este campo é obrigatório</span>}
31.         </div>
32.
33.         <button type="submit">Enviar</button>
34.             </form>
35.         </>
            );
36.         // Esta parte do código fecha o formulário e retorna o restante do TSX. Um botão "Enviar" é renderizado dentro do formulário, que será usado para submeter o formulário quando clicado.
37.     }
38.     export default Exemplo01;
```

VALIDAÇÃO RÁPIDA E PRECISA

- ✓ **Introdução ao Hook Form**
  - ✓ Exemplo 01 – Resultado



VALIDAÇÃO RÁPIDA E PRECISA

- **Spread Operator ...**

  - O **...** que usamos ao passar **register** dentro dos *inputs* é chamado de ***spread operator*** (ou "operador de espalhamento") no JavaScript.

  - Ele espalha as propriedades de um objeto dentro de outro objeto ou estrutura.

  - No caso do React Hook Form, register("nome") retorna um objeto com diversas propriedades:

```
{
    name: "nome",
    ref: ...,
    onChange: ...,
    onBlur: ...
}
```

VALIDAÇÃO RÁPIDA E PRECISA

- **Spread Operator ...**

  - Quando usamos

```
<input { ...register("nome") } />
```

  - É o mesmo que usar

```
<input
        name="nome"
        ref={ register("nome").ref }
        onChange={ register("nome").onChange }
        onBlur={ register("nome").onBlur }
/>
```

VALIDAÇÃO RÁPIDA E PRECISA

- **Spread Operator ...**

  - Além de espalhar propriedades, ele pode ser usado para copiar e mesclar *arrays* e *objetos*:

    - **Copiar Arrays**

```
const numeros = [1, 2, 3];
const novosNumeros = [...numeros, 4, 5]; // [1, 2, 3, 4, 5]
```

    - **Copiar e modificar objetos**

```
const pessoa = { nome: "João", idade: 25 };
const novaPessoa = { ...pessoa, cidade: "São Paulo" };
console.log(novaPessoa); // { nome: "João", idade: 25, cidade: "São Paulo" }
```

VALIDAÇÃO RÁPIDA E PRECISA

- **Spread Operator ...**

  - **Exemplo**

```
<input
    {...register("nome", {
        required: "Nome obrigatório",
        minLength: { value: 3, message: "Mínimo de 3 caracteres" }
    })}
    placeholder="Digite seu nome"
/>
{errors.nome && <p>{errors.nome.message}</p>}
```

VALIDAÇÃO RÁPIDA E PRECISA

- **Introdução ao Hook Form**
  - Exemplo 02 – Vamos criar um novo formulário, agora com outros tipos de campos para validação:
  - Neste exemplo, vamos validar os seguintes campos:
    - um campo para o nome
    - um campo para o email,
    - um campo para a senha,
    - um campo para confirmação de senha,
    - um campo para seleção do país,
    - um campo para selecionar gênero e
    - e um campo para aceitar os termos.

VALIDAÇÃO RÁPIDA E PRECISA

- ✓ **Introdução ao Hook Form**
  - ✓ **Exemplo 02 –** Parte 01

```
1.  //src/Exemplo02/Exemlo02.tsx
2.  import{ useForm, type SubmitHandler } from "react-hook-form"
3.
4.  type FormValues ={
5.    nome: string
6.    email: string
7.    senha: string
8.    confirmarSenha: string
9.    genero: string
10.   pais: string
11.   termos: boolean
12. }
13.
```

VALIDAÇÃO RÁPIDA E PRECISA

- ✓ **Introdução ao Hook Form**
  - ✓ **Exemplo 02 –** Parte 02

```
14. const Exemplo02 = () =>{
15.   const{
16.     register,
17.     handleSubmit,
18.     watch,
19.     formState:{ errors }
20.   } = useForm<FormValues>()
21.
22.   const onSubmit: SubmitHandler<FormValues> = (data) =>{
23.     console.log("Dados enviados:", data)
24.   }
25.
26.   // Observa o campo senha
27.   const senha = watch("senha")
28.
```

VALIDAÇÃO RÁPIDA E PRECISA

- ✓ **Introdução ao Hook Form**
  - ✓ **Exemplo 02 –** Parte 03

```
29.  return (
30.    <>
31.      <h1>Formulário com React Hook Form</h1>
32.      <h2>Exemplo 02 - Formulário mais completo com React Hook Form</h2>
33.      <form onSubmit={handleSubmit(onSubmit)}>
34.        {/* Nome */}
35.        <div>
36.            <label>Nome</label>
37.            <input
38.                type="text"
39.                { ...register("nome",{ required: "O nome é obrigatório" }) }
40.            />
41.            { errors.nome && <span style={ { color: "red" } }>{ errors.nome.message }</span> }
42.        </div>
43.        <br />
44.
```

VALIDAÇÃO RÁPIDA E PRECISA

- ✓ **Introdução ao Hook Form**
  - ✓ **Exemplo 02 –** Parte 04

```
45.          {/* Email */}
46.         <div>
47.            <label>Email</label>
48.            <input
49.              type="email"
50.              { ...register("email",{
51.                  required: "O email é obrigatório",
52.                  pattern:{
53.                      value: /^[^\s@]+@[^\s@]+\.[^\s@]+$/,
54.                      message: "Digite um email válido"
55.                  }
56.              }) }
57.            />
58.            { errors.email &&<span style={ { color: "red" } }>{ errors.email.message }</span> }
59.         </div>
60.         <br />
```

VALIDAÇÃO RÁPIDA E PRECISA

- ✓ **Introdução ao Hook Form**
  - ✓ **Exemplo 02 –** Parte 05

```
61.
62.         {/* Senha */}
63.         <div>
64.           <label>Senha</label>
65.           <input
66.               type="password"
67.               { ...register("senha",{
68.                   required: "A senha é obrigatória",
69.                   minLength: { value: 6, message: "Mínimo de 6 caracteres" },
70.                   validate: value =>
71.                   !/[A-Z]/.test(value) ? "Deve conter uma letra maiúscula" :
72.                   !/[a-z]/.test(value) ? "Deve conter uma letra minúscula" :
73.                   !/\d/.test(value) ? "Deve conter um número" :
74.                   !/[^A-Za-z0-9]/.test(value) ? "Deve conter um caractere especial" :
75.                   true
76.               })}
```

VALIDAÇÃO RÁPIDA E PRECISA

- ✓ **Introdução ao Hook Form**
  - ✓ **Exemplo 02 –** Parte 06

```
77.          />
78.        { errors.senha && <span style={ { color: "red" } }>{ errors.senha.message }</span> }
79.        </div>
80.        <br />
81.
82.        {/* Confirmação de Senha */}
83.        <div>
84.          <label>Confirme a Senha</label>
85.          <input
86.              type="password"
87.              { ...register("confirmarSenha",{
88.                  required: "Confirme a senha",
89.                  validate: (value) =>
90.                      value === senha || "As senhas não coincidem"
91.              }) }
92.          />
```

VALIDAÇÃO RÁPIDA E PRECISA

- ✓ **Introdução ao Hook Form**
  - ✓ **Exemplo 02 –** Parte 07

```
93.          { errors.confirmarSenha && (
94.              <span style={ { color: "red" } }>{ errors.confirmarSenha.message }</span>
95.          ) }
96.        </div>
97.        <br />
98.
99.        {/* Select */}
100.       <div>
101.         <label>País</label>
102.         <select { ...register("pais",{ required: "Selecione um país" }) }>
103.             <option value="">-- Escolha --</option>
104.             <option value="brasil">Brasil</option>
105.             <option value="portugal">Portugal</option>
106.             <option value="eua">EUA</option>
107.         </select>
```

VALIDAÇÃO RÁPIDA E PRECISA

- ✓ **Introdução ao Hook Form**
  - ✓ **Exemplo 02 –** Parte 08

```
108.          { errors.pais &&<span style={ { color: "red" } }>{ errors.pais.message }</span> }
109.        </div>
110.        <br />
111.
112.        {/* Radio */}
113.        <div>
114.          <span>Gênero
115.              <label>
116.                  <input
117.                      type="radio"
118.                      value="masculino"
119.                      { ...register("genero",{ required: "Escolha um gênero" }) }
120.                  />Masculino
121.              </label>
```

VALIDAÇÃO RÁPIDA E PRECISA

- ✓ **Introdução ao Hook Form**
  - ✓ **Exemplo 02 –** Parte 09

```
122.                <label>
123.                    <input
124.                        type="radio"
125.                        value="feminino"
126.                        { ...register("genero",{ required: "Escolha um gênero" }) }
127.                    />Feminino
128.                </label>
129.                { errors.genero &&<span style={ { color: "red" } }>{ errors.genero.message}</span> }
130.            </span>
131.        </div>
132.        <br />
133.
```

VALIDAÇÃO RÁPIDA E PRECISA

- ✓ **Introdução ao Hook Form**
  - ✓ **Exemplo 02 –** Parte 10

```
134.        {/* Checkbox */}
135.        <div>
136.            <label>
137.                <input
138.                    type="checkbox"
139.                    { ...register("termos",{ required: "Você deve aceitar os termos" }) }
140.                />Aceito os termos de uso
141.            </label>
142.            { errors.termos && <span style={ { color: "red" } }>{ errors.termos.message }</span> }
143.        </div>
144.        <br />
145.
```

VALIDAÇÃO RÁPIDA E PRECISA

- **Introdução ao Hook Form**
  - **Exemplo 02 –** Parte 11

```
146.          {/* Botão */}
147.          <button type="submit">Enviar</button>
148.        </form>
149.      </>
150.  )
151.  }
152.  export default Exemplo02
```

VALIDAÇÃO RÁPIDA E PRECISA

✓ Introdução ao Hook Form

✓ Exemplo 02 – Analisando linha a linha o arquivo Exemplo02.tsx **(vamos analisar somente o código que inserimos a partir do Exemplo01)**

```
14.  const Exemplo02 = () =>{
15.    const{
16.      register,
17.      handleSubmit,
18.      watch,
19.      formState:{ errors }
20.    } = useForm<FormValues>()
21.
22.    const onSubmit: SubmitHandler<FormValues> = (data) =>{
23.      console.log("Dados enviados:", data)
24.    }
25.
26.    // Observa o campo senha
27.    const senha = watch("senha")
28.
```

// A linha const { register, handleSubmit, watch, formState: { errors } } = useForm<FormValues>(); **desestrutura** o retorno do hook useForm do React Hook Form, que é usado para gerenciar formulários em React.

// A função **register** é utilizada para vincular os campos de entrada ao gerenciamento de estado do formulário.

// **HandleSubmit** lida com a submissão do formulário, chamando a função de **callback** fornecida se a validação for bem-sucedida.

// **Watch** permite observar e acompanhar os valores dos campos em tempo real, e **formState**: { errors } acessa os erros de validação do formulário, que podem ser usados para exibir mensagens de erro quando um campo não é preenchido corretamente.

// O tipo **FormValues** define a estrutura dos dados do formulário.

// Utilizamos a função **watch** do React Hook Form para observar o valor do campo password em tempo real. Sempre que o usuário digitar ou alterar a senha no campo correspondente, a variável senha será atualizada automaticamente com o valor atual desse campo. Isso permite que você use esse valor em outras partes do código, como para validações em tempo real ou ajustes de comportamento, sem precisar usar um estado separado.

VALIDAÇÃO RÁPIDA E PRECISA

- ✓ **Introdução ao Hook Form**
  - ✓ Exemplo 02 – Analisando linha a linha o arquivo Exemplo02.tsx **(vamos analisar somente o código que inserimos a partir do Exemplo01)**

```
45.          {/* Email */}
46.         <div>
47.            <label>Email</label>
48.            <input
49.              type="email"
50.              { ...register("email",{
51.                  required: "O email é obrigatório",
52.                  pattern:{
53.                      value: /^[^\s@]+@[^\s@]+\.[^\s@]+$/,
54.                      message: "Digite um email válido"
55.                  }
56.              }) }
57.            />
58.            { errors.email &&<span style={ { color: "red" } }>{ errors.email.message }</span> }
59.         </div>
60.         <br />
```

// A função register do React Hook Form para vincular o campo de entrada de e-mail ao gerenciamento de formulário, aplicando validações.

// A validação required exige que o campo seja preenchido, com a mensagem de erro "E-mail obrigatório" caso não seja preenchido.

// A validação pattern verifica se o valor digitado corresponde a um formato de e-mail válido (usando uma expressão regular), e se o e-mail for inválido, a mensagem "E-mail incorreto!" será exibida.

// O operador {...} espalha as propriedades retornadas pelo register para o campo de entrada, conectando-o ao sistema de controle de estado do formulário.

VALIDAÇÃO RÁPIDA E PRECISA

- ✓ **Introdução ao Hook Form**
  - ✓ Exemplo 02 – Analisando linha a linha o arquivo page.tsx **(vamos analisar somente o código que inserimos a partir do Exemplo01)**

```
61.
62.         {/* Senha */}
63.         <div>
64.           <label>Senha</label>
65.           <input
66.               type="password"
67.               { ...register("senha",{
68.                   required: "A senha é obrigatória",
69.                   minLength: { value: 6, message: "Mínimo de 6 caracteres" },
70.                   validate: value =>
71.                   !/[A-Z]/.test(value) ? "Deve conter uma letra maiúscula" :
72.                   !/[a-z]/.test(value) ? "Deve conter uma letra minúscula" :
73.                   !/\d/.test(value) ? "Deve conter um número" :
74.                   !/[^A-Za-z0-9]/.test(value) ? "Deve conter um caractere especial" :
75.                   true
76.               })}
```

// O trecho de código registra o campo de senha no formulário com várias validações.
A validação **required** exige que o campo seja preenchido, com a mensagem "Senha obrigatória" se vazio.
A validação **minLength** exige no mínimo 6 caracteres, com a mensagem "Mínimo de 6 caracteres" caso não seja atendido.
A função **validate** aplica múltiplas validações personalizadas para garantir que a senha contenha pelo menos uma letra maiúscula, uma letra minúscula, um número e um caractere especial, retornando mensagens específicas para cada falha.
O operador {...} espalha as propriedades do register para o campo de entrada, conectando-o ao gerenciamento de estado do React Hook Form.

VALIDAÇÃO RÁPIDA E PRECISA

- ✓ **Introdução ao Hook Form**
  - ✓ Exemplo 02 – Resultado

hookforms

localhost:5173/exemplo02

- HOME
- Exemplo 01
- Exemplo 02

Formulário com React Hook Form

Exemplo 02 - Formulário mais completo com React Hook Form

Nome

Email

Senha

Confirme a Senha

País -- Escolha --

Gênero ○ Masculino ○ Feminino

☐ Aceito os termos de uso

Enviar

Estado inicial

hookforms

localhost:5173/exemplo02

- HOME
- Exemplo 01
- Exemplo 02

Formulário com React Hook Form

Exemplo 02 - Formulário mais completo com React Hook Fo

Nome O nome é obrigatório

Email O email é obrigatório

Senha A senha é obrigatória

Confirme a Senha Confirme a senha

País -- Escolha -- Selecione um país

Gênero ○ Masculino ○ Feminino Escolha um gênero

☐ Aceito os termos de uso Você deve aceitar os termos

Enviar

Clicar o botão 'Enviar' com os campos vazios

VALIDAÇÃO RÁPIDA E PRECISA

- ✓ **Introdução ao Hook Form**
  - ✓ Exemplo 02 – Resultado

Campos não preenchidos corretamente e suas respectivas mensagens de erro personalizadas.

2

VITE – REACT HOOK FORM

CONSUMINDO API EXTERNA

VALIDAÇÃO RÁPIDA E PRECISA – CONSUMINDO API

- ✓ O consumo de uma API externa em uma aplicação web envolve a troca de dados entre a sua aplicação e um serviço externo, geralmente por meio de requisições HTTP.
- ✓ Esse processo é essencial para acessar funcionalidades ou informações que não estão armazenadas localmente na sua aplicação.
- ✓ APIs externas podem fornecer dados de diversos tipos, como informações de usuários, dados financeiros, previsões do tempo, e, no caso do exemplo anterior, dados de endereços a partir de um CEP.

VALIDAÇÃO RÁPIDA E PRECISA – CONSUMINDO API

- **Como Funciona o Consumo de uma API**

  - **Requisição HTTP**
    - O consumo de uma API externa começa com uma requisição HTTP (GET, POST, PUT, DELETE, etc.) enviada para o servidor que hospeda a API. No React, geralmente usamos funções como **fetch, axios**, ou outros métodos para fazer essas requisições.

  - **Autenticação (se necessário)**
    - Algumas APIs externas requerem autenticação. Isso pode ser feito usando tokens de acesso (como tokens JWT) ou chaves de API, que são enviadas junto com a requisição para garantir que o cliente tem permissão para acessar os dados.

  - **Resposta da API**
    - A API retorna uma resposta, geralmente em formato JSON ou XML.
    - A resposta da API pode incluir um status HTTP que indica o sucesso ou falha da requisição (ex: 200 OK, 404 Not Found, 500 Internal Server Error).

VALIDAÇÃO RÁPIDA E PRECISA – CONSUMINDO API

- **Como Funciona o Consumo de uma API** (continuação)
  - **Tratamento de Dados**
    - Após a resposta ser recebida, a aplicação pode processar os dados conforme necessário. Isso pode incluir, por exemplo, preencher os campos de um formulário com os dados retornados ou realizar algum cálculo com os dados.
  - **Exibição na Interface**
    - Após o tratamento dos dados, a aplicação exibe essas informações na interface do usuário (UI). Isso pode ser feito, por exemplo, com o uso de estados em React (useState) ou outras bibliotecas de gerenciamento de estado.

VALIDAÇÃO RÁPIDA E PRECISA – CONSUMINDO API

- **Considerações importantes**
  - **Erro de Requisição:**
    - É importante tratar erros de requisição corretamente. Caso a API não esteja acessível ou a requisição falhe por algum motivo, a aplicação deve fornecer uma mensagem de erro ou uma resposta amigável para o usuário.
  - **Performance:**
    - Requisições externas podem afetar a performance da aplicação, especialmente se forem lentas ou bloquearem o fluxo da interface. Nesse caso, o uso de loading states ou até mesmo cache de dados pode melhorar a experiência do usuário.
  - **Limitações de API:**
    - Algumas APIs impõem limites de requisições, o que significa que você só pode fazer um número determinado de requisições dentro de um intervalo de tempo. Certifique-se de entender essas limitações para evitar erros ou bloqueios.

VALIDAÇÃO RÁPIDA E PRECISA – CONSUMINDO API

- **Considerações importantes** (continuação)

  - **CORS (Cross-Origin Resource Sharing):**

    - APIs externas podem ter restrições de CORS, que impedem que sua aplicação faça requisições de domínios diferentes. Nesse caso, você pode precisar configurar um servidor intermediário ou verificar se a API oferece suporte a CORS.

    - O consumo de APIs externas é uma prática essencial no desenvolvimento de aplicações modernas, permitindo que elas se integrem com outros serviços e tragam informações e funcionalidades adicionais de maneira dinâmica.

VALIDAÇÃO RÁPIDA E PRECISA – CONSUMINDO API

- **Introdução ao Hook Form – Consumindo API externa**

  - Vamos modificar o formulário criado na aula passada e inserir mais alguns campos, como por exemplo: CEP, RUA, NUMERO, COMPLEMENTO, BAIRRO, CIDADE, ESTADO e ESTADO.

  - As APIs (Application Programming Interface ou Interfaces de Programação de Aplicações) são fundamentais para a comunicação entre diferentes sistemas e para fornecer acesso a funcionalidades específicas de uma aplicação. No ambiente TypeScript, criar e consumir APIs é uma tarefa acessível e eficiente, permitindo a construção de serviços web poderosos e escaláveis.

  - No nosso, caso não vamos construir, ainda, mas vamos sim, consumir uma API.

VALIDAÇÃO RÁPIDA E PRECISA – CONSUMINDO API

- ✓ **Introdução ao Hook Form – Consumindo API externa**
  - ✓ **Exemplo 03** – Parte 01 – Vamos modificar o Exemplo02

```
1.  // Importa o React e o useEffect para gerenciar efeitos colaterais.
2.  import { useEffect, useCallback } from "react";
3.
4.  // Importa o useForm, um hook para manipulação de formulários no React.
5.  import { useForm } from "react-hook-form";
6.
7.  //  Define a interface FormValues, garantindo que o formulário siga essa estrutura.
8.  //  Cada campo é do tipo string, especificando os dados que serão manipulados.
9.  type FormValues = {
10.     firstName : string;
11.     lastName : string;
12.     email : string;
13.     senha : string;
14.     confpassword : string;
```

VALIDAÇÃO RÁPIDA E PRECISA – CONSUMINDO API

- ✓ **Introdução ao Hook Form – Consumindo API externa**
  - ✓ **Exemplo 03** – Parte 02 – Vamos modificar o Exemplo02

```
    cep : string;
    rua : string;
    bairro : string;
    cidade : string;
    estado : string;
    numero : string;
};

// Cria um componente funcional chamado Exemplo03.
const Exemplo03 = () => {
    // Inicialização do Hook useForm
    // register → Registra os campos do formulário para que o React Hook Form os controle.
    // handleSubmit → Gerencia o envio do formulário.
    // setValue → Permite definir valores nos campos manualmente.
```

VALIDAÇÃO RÁPIDA E PRECISA – CONSUMINDO API

- ✓ **Introdução ao Hook Form – Consumindo API externa**
  - ✓ **Exemplo 03** – Parte 03 – Vamos modificar o Exemplo02

```
29.     // setFocus → Define o foco em um campo específico.
30.     // watch → Observa os valores de um campo em tempo real.
31.     // formState: {  errors  } → Captura os erros de validação do formulário.
32.     const { register, handleSubmit, setValue, setFocus, watch, formState: { errors } } = useForm<FormValues>();
33.
34.     // Captura de valores com watch
35.     // watch("cep") → Monitora o campo CEP para buscar o endereço automaticamente.
36.     const cep = watch("cep");
37.
38.     // watch("password") → Monitora a senha para validação na confirmação.
39.     const senha = watch("senha");
40.
41.     // Função para Buscar Endereço pelo CEP
42.     const buscarEndereco = useCallback(async (cep: string) => {
```

VALIDAÇÃO RÁPIDA E PRECISA – CONSUMINDO API

- ✓ **Introdução ao Hook Form – Consumindo API externa**
  - ✓ **Exemplo 03** – Parte 04 – Vamos modificar o Exemplo02

```
43.         if (cep.length === 8) {
44.             try {
45.                 const response = await fetch(`https://viacep.com.br/ws/${ cep }/json/`);
46.                 const data = await response.json();
47.                 if (data.erro) {
48.                     alert("CEP não encontrado!");
49.                 } else {
50.                     setValue("rua", data.logradouro);
51.                     setValue("bairro", data.bairro);
52.                     setValue("cidade", data.localidade);
53.                     setValue("estado", data.uf);
54.                     setFocus("numero");
55.                 }
56.             } catch (error) {
```

VALIDAÇÃO RÁPIDA E PRECISA – CONSUMINDO API

- ✓ **Introdução ao Hook Form – Consumindo API externa**
  - ✓ **Exemplo 03** – Parte 05 – Vamos modificar o Exemplo02

```
                console.error("Erro ao buscar endereço:", error);
                alert("Erro ao buscar o endereço.");
            }
        }
    }, [setValue, setFocus]);
    // Verifica se o CEP tem 8 dígitos antes de buscar.
    // Faz uma requisição à API do ViaCEP para obter os dados do endereço.
    // Se o CEP for inválido, exibe um alerta "CEP não encontrado!".
    // Se o CEP for válido, preenche os campos de rua, bairro, cidade e estado usando setValue().
    // Foca automaticamente no campo "Número" com setFocus("numero").

    // useEffect para Monitorar o CEP
    useEffect(() => {
        if (cep && cep.length === 8) {
```

VALIDAÇÃO RÁPIDA E PRECISA – CONSUMINDO API

- ✓ **Introdução ao Hook Form – Consumindo API externa**
  - ✓ **Exemplo 03** – Parte 06 – Vamos modificar o Exemplo02

```
71.             buscarEndereco(cep);
72.         }
73.     }, [cep,buscarEndereco]);
74.     // Sempre que o CEP for alterado, e tiver 8 caracteres, chama buscarEndereco().
75.
76.     // Função para Submissão do Formulário
77.     const onSubmit = (data: FormValues) => {
78.         console.log(data);
79.     };
80.     // Exibe os dados no console quando o formulário for enviado.
81.
82.     return (
83.         <>
84.
```

VALIDAÇÃO RÁPIDA E PRECISA – CONSUMINDO API

- ✓ **Introdução ao Hook Form – Consumindo API externa**
  - ✓ **Exemplo 03** – Parte 07 – Vamos modificar o Exemplo02

```
85.             <h1>Formulário com React Hook Form</h1>
86.             <h2>Exemplo 03 - Formulário Consumindo API externa</h2>
87.
88.             { /* Estrutura do Formulário */ }
89.             <form onSubmit={ handleSubmit(onSubmit) }>
90.                 { /* O formulário chama handleSubmit(onSubmit) para validar e enviar os dados.
91.                     Nome e Sobrenome - Cada campo tem validações e exibe mensagens de erro
92.                     caso o usuário não preencha corretamente.
 93                     O campo firstName exige um mínimo de 3 caracteres.
94.                     Se houver erro, exibe a mensagem errors.firstName.message.
95.                 */ }
96.                 <div>
97.                     <label>Primeiro Nome:</label>
98.                     <input
99.                         { ...register("firstName", {
```

VALIDAÇÃO RÁPIDA E PRECISA – CONSUMINDO API

- **Introdução ao Hook Form – Consumindo API externa**
  - **Exemplo 03** – Parte 08 – Vamos modificar o Exemplo02

```
                            required: "Campo Nome obrigatório!" ,
                            minLength:{ value: 3, message: "Mínimo de 3 caracteres!" }
                        }) }
                    />
                    { errors.firstName && <span>{ errors.firstName.message }</span> }
                </div>

                <div>
                    <label>Sobrenome:</label>
                    <input
                        { ...register("lastName", {
                            required: "Campo Sobrenome obrigatório!" ,
                        }) }
                    />
```

VALIDAÇÃO RÁPIDA E PRECISA – CONSUMINDO API

- ✓ **Introdução ao Hook Form – Consumindo API externa**
  - ✓ **Exemplo 03** – Parte 09 – Vamos modificar o Exemplo02

```
114.                    { errors.lastName && <span>{ errors.lastName.message }</span> }
115.                </div>
116.
117.                <div>
118.                   { /*  EMAIL
119.                   Valida se o e-mail está no formato correto (\S+@\S+\.\S+). */ }
120.                    <label>Email:</label>
121.                    <input
122.                        { ...register("email", {
123.                            required: "O email é obrigatório",
124.                            pattern: {
125.                              value: /^[^\s@]+@[^\s@]+\.[^\s@]+$/,
126.                              message: "Digite um email correto"
127.                            }
```

VALIDAÇÃO RÁPIDA E PRECISA – CONSUMINDO API

- ✓ **Introdução ao Hook Form – Consumindo API externa**
  - ✓ **Exemplo 03** – Parte 10 – Vamos modificar o Exemplo02

```
                        }) }
                        placeholder="E-mail"
                    />
                    { errors.email && <span>{ errors.email.message }</span> }
                </div>

                <div>
                    { /*
                        Senha e Confirmação
                        Valida se a senha contém letra maiúscula, minúscula, número e caractere especial.
                        Cada erro é tratado separadamente usando ternário (? :).
                        Valida se a confirmação de senha (confpassword) é igual à senha original.
                    */ }
                    <label>Senha:</label>
```

VALIDAÇÃO RÁPIDA E PRECISA – CONSUMINDO API

- ✓ **Introdução ao Hook Form – Consumindo API externa**
  - ✓ **Exemplo 03** – Parte 11 – Vamos modificar o Exemplo02

```
142.                    <input
143.                        type="password"
144.                        {   ...register("senha", {
145.                            required: "Senha obrigatória",
146.                            minLength: {  value: 6, message: "Mínimo de 6 caracteres"  },
147.                            validate: value =>
148.                            !/[A-Z]/.test(value) ? "Deve conter uma letra maiúscula" :
149.                            !/[a-z]/.test(value) ? "Deve conter uma letra minúscula" :
150.                            !/\d/.test(value) ? "Deve conter um número" :
151.                            !/[^A-Za-z0-9]/.test(value) ? "Deve conter um caractere especial" :
152.                            true
153.                        }) }
154.                        />
155.                    { errors.senha && <span>{ errors.senha.message }</span> }
```

VALIDAÇÃO RÁPIDA E PRECISA – CONSUMINDO API

- **Introdução ao Hook Form – Consumindo API externa**
  - **Exemplo 03** – Parte 12 – Vamos modificar o Exemplo02

```
156.                </div>
157.
158.                <div>
159.                    <label>Confirmação da Senha:</label>
160.                    <input
161.                        type="password"
162.                        { ...register("confpassword", {
163.                            required: "Confirmação obrigatória",
164.                            validate: value => value === senha || "As senhas devem ser iguais"
165.                         }) }
166.                        placeholder="Confirmar Senha"
167.                    />
168.                    { errors.confpassword && <span>{ errors.confpassword.message }</span> }
169.                </div>
```

VALIDAÇÃO RÁPIDA E PRECISA – CONSUMINDO API

- **Introdução ao Hook Form – Consumindo API externa**
  - **Exemplo 03** – Parte 13 – Vamos modificar o Exemplo02

```
170.
171.                <div>
172.                    { /* CEP e Endereço */ }
173.                    <label>CEP:</label>
174.                    <input
175.                        {...register("cep", {
176.                            required: "CEP é obrigatório",
177.                            validate: value =>
178.                                value.length === 8 && /^[0-9]{8}$/.test(value)
179.                                ? true
180.                             }
181.                         }) }
182.                        maxLength={ 8 }
183.                    />
```

VALIDAÇÃO RÁPIDA E PRECISA – CONSUMINDO API

- ✓ **Introdução ao Hook Form – Consumindo API externa**
  - ✓ **Exemplo 03** – Parte 14 – Vamos modificar o Exemplo02

```
184.                    { errors.cep && <span>{ errors.cep.message }</span> }
185.                </div>
186.
187.                <div>
188.                    { /* Os campos de rua, bairro, cidade e estado são preenchidos
189.                    automaticamente e são somente leitura */ }
190.                    <label>Rua:</label>
191.                    <input { ...register("rua") } readOnly />
192.                </div>
193.
194.                <div>
195.                    <label>Bairro:</label>
196.                    <input { ...register("bairro") } readOnly />
197.                </div>
```

VALIDAÇÃO RÁPIDA E PRECISA – CONSUMINDO API

- **Introdução ao Hook Form – Consumindo API externa**
  - **Exemplo 03** – Parte 15 – Vamos modificar o Exemplo02

```
198.
199.                <div>
200.                    <label>Cidade:</label>
201.                    <input { ...register("cidade") } readOnly />
202.                </div>
203.
204.                <div>
205.                    <label>Estado:</label>
206.                    <input { ...register("estado") } readOnly />
207.                </div>
208.
209.                <div>
210.                    <label>Número:</label>
211.                    <input
```

VALIDAÇÃO RÁPIDA E PRECISA – CONSUMINDO API

- **Introdução ao Hook Form – Consumindo API externa**
  - **Exemplo 03** – Parte 16 – Vamos modificar o Exemplo02

```
                        { ...register("numero", {  required: "Número é obrigatório"  }) }
                    />
                    { errors.numero && <span>{ errors.numero.message }</span> }
                </div>

                <button type="submit">Cadastrar</button>
            </form>
        </>
    );
};

export default Exemplo03;
```

VALIDAÇÃO RÁPIDA E PRECISA

- ✓ **Introdução ao Hook Form – Consumindo API externa**
  - ✓ Exemplo 02 – Resultado

hookforms | localhost:5173/exemplo03

- HOME
- Exemplo 01 - FORM Simples
- Exemplo 02 - FORM Completo
- Exemplo 03 - Consumindo API

Formulário com React Hook Form

Exemplo 03 - Formulário Consumindo API externa

Primeiro Nome: Campo Nome obrigatório!
Sobrenome: Campo Sobrenome obrigatório!
Email: E-mail O email é obrigatório
Senha: Senha obrigatória
Confirmação da Senha: Confirmar Senha Confirmação obrigatória
CEP: 01310930
Rua: Avenida Paulista
Bairro: Bela Vista
Cidade: São Paulo
Estado: SP
Número: Número é obrigatório
Cadastrar

Campos não preenchidos corretamente e suas respectivas mensagens de erro.

hookforms | Exemplos de liderança e cultura | localhost:5173/exemplo03

- HOME
- Exemplo 01 - FORM Simples
- Exemplo 02 - FORM Completo
- Exemplo 03 - Consumindo API

Formulário com React Hook Form

Exemplo 03 - Formulário Consumindo API externa

Primeiro Nome:
Sobrenome:
Email: E-mail
Senha:
Confirmação da Senha: Confirmar Senha
CEP:
Rua:
Bairro:
Cidade:
Estado:
Número:
Cadastrar

Estado inicial

3

VITE – REACT HOOK FORM

EXERCÍCIOS

VALIDAÇÃO RÁPIDA E PRECISA – CONSUMINDO API

- **Introdução ao Hook Form**

  - **Exercício 1 – Fácil (Cadastro simples com validação)**
    - Criar um formulário de cadastro com nome e email usando **React Hook Form**.
    - **Regras**:
      - Nome obrigatório, mínimo de 3 caracteres.
      - Email obrigatório, precisa ser válido (teste@exemplo.com).
      - Mostrar mensagem de erro abaixo de cada campo.
      - **Dica**: use register, handleSubmit e formState.errors.

VALIDAÇÃO RÁPIDA E PRECISA – CONSUMINDO API

- ✓ **Introdução ao Hook Form**

  - ✓ **Exercício 2 – Intermediário (Busca de Pokémon com API do PokéAPI)**
    - ✓ Criar um formulário com um campo **nomeDoPokemon**.
    - ✓ Ao enviar o formulário, consumir a API PokéAPI → https://pokeapi.co/api/v2/pokemon/{nome}
    - ✓ Mostrar
      - ✓ o nome,
      - ✓ altura,
      - ✓ Peso
      - ✓ a imagem oficial do Pokémon.
    - ✓ Se não existir, exibir mensagem de erro.
    - ✓ **Dica**: Use onSubmit para chamar fetch, guarde os dados em um useState e renderize abaixo do formulário.

VALIDAÇÃO RÁPIDA E PRECISA – CONSUMINDO API

- **Introdução ao Hook Form**

  - **Exercício 3 – Intermediário (Escolha de País e Consulta de API)**
    - Criar um formulário com um campo de **busca por país** e, ao digitar o nome, trazer informações usando a API REST Countries.
    - **Regras**:
      - Campo obrigatório: país.
      - Ao digitar e enviar, buscar na API https://restcountries.com/v3.1/name/{nome}.
      - Preencher automaticamente:
        - Capital
        - População
        - Continente
        - Bandeira (imagem).
      - **Dica**: Assim você pratica **watch**, **setValue**, **useEffect**, **fetch**, mas em um contexto diferente do CEP

VALIDAÇÃO RÁPIDA E PRECISA – CONSUMINDO API

- **Introdução ao Hook Form**

  - **Exercício 4 – Difícil (Consulta do clima com OpenWeather)**
    - Criar um formulário onde o usuário digita uma cidade e recebe os dados da API OpenWeather.
    - **Regras**:
      - Campo obrigatório: cidade.
      - Ao enviar, mostrar:
        - Temperatura atual
        - Descrição do clima
        - Ícone correspondente (ex.: , ,)
      - Se a cidade não for encontrada, exibir erro.
    - **Dica**: Você precisa gerar uma API Key gratuita no site do OpenWeather e usar no fetch

Referências Bibliográficas

BANKS, Antony; PORTEOUS, Andrew. Beginning React. New York: Apress, 2022.

BUECHELER, John. React 18 Design Patterns and Best Practices. Birmingham: Packt Publishing, 2023.

BURGESS, Matt. Why would I use React Hook Form? Medium, 2020. Disponível em: <https://mattburgess.medium.com/why-would-i-use-react-hook-form-8ae7b905953e>. Acesso em: 30 de setembro de 2025.

FRANA, Matej. React Hook Form – The Best Form Library? Dev.to, 2021. Disponível em: <https://dev.to/matfrana/react-hook-form-the-best-form-library-1o1l>. Acesso em: 30 de setembro de 2025.

FREECODECAMP. React Hook Form Crash Course. FreeCodeCamp, 2022. Disponível em: <https://www.freecodecamp.org/news/react-hook-form-tutorial-with-examples/>. Acesso em: 30 de setembro de 2025.

FULLSTACK.IO. Fullstack React: The Complete Guide to ReactJS and Friends. Fullstack.io, 2020.

GITHUB. react-hook-form/documentation – Docs Repository. Disponível em: <https://github.com/react-hook-form/documentation>. Acesso em: 30 de setembro de 2025.

GITHUB. react-hook-form/react-hook-form – Official Repository. Disponível em: <https://github.com/react-hook-form/react-hook-form>. Acesso em: 30 de setembro de 2025.

GRIDER, Stephen. Modern React with Redux. 3. ed. Boston: Pearson Addison-Wesley, 2021.

JOHNSON, Adam. Build Robust and Scalable Web Forms with React Hook Form. 1. ed. [S. l.]: Independently published, 2024.

LERNA, David. React – Up & Running: Building Web Applications. Sebastopol: O’Reilly Media, 2020.

LOGROÑO, Juan. React Hook Form – Complete Tutorial with Zod. YouTube, 2023. Disponível em: <https://www.youtube.com/watch?v=cc_xmawJ8Kg>. Acesso em: 30 de setembro de 2025.

MARDIROSOV, Alex. React 18 Essentials. Birmingham: Packt Publishing, 2022.

MCGINNIS, Alex. React Cookbook: Recipes for Mastering the React Framework. Sebastopol: O’Reilly Media, 2021.

Referências Bibliográficas

MDN WEB DOCS. Client-side form validation. Disponível em: <https://developer.mozilla.org/en-US/docs/Learn/Forms/Form_validation>. Acesso em: 30 de setembro de 2025.

REACT HOOK FORM. API Reference – React Hook Form. Disponível em: <https://react-hook-form.com/api>. Acesso em: 30 de setembro de 2025.

REACT HOOK FORM. Documentação oficial. Disponível em: <https://www.react-hook-form.com/>. Acesso em 30 de setembro de 2025.

REACT HOOK FORM. Get Started – React Hook Form. Disponível em: https://react-hook-form.com/get-started. Acesso em: 30 de setembro de 2025.

REACTJS. React Official Documentation. Disponível em: https://react.dev. Acesso em: 30 de setembro de 2025.

REHMAN, U. A. Web Forms with React: Build Robust and Scalable Forms with React Hook Form. New York: Apress, 2025.

ROYLE, Carl. React Explained. San Francisco: Pragmatic Bookshelf, 2020.

TYPESCRIPT SUPPORT. Disponível em <https://react-hook-form.com/ts#UseFormProps> . Acesso em 30 de setembro de 2025.

YUP. Disponível em <https://www.npmjs.com/package/yup> . Acesso em 30 de setembro de 2025.

ZHANG, Michele. Learning React: Modern Patterns for Developing React Apps. 2. ed. Sebastopol: O’Reilly Media, 2021.

Dúvidas?

Críticas?

Sugestões?

Ameaças?